第04堂｜跨平台 Web 數據擷取與異質系統整合

學習目標

完成本堂課後，學員將能夠：

- 說明 Web 自動化與 Web 爬蟲的差異及各自的應用場景
- 使用瀏覽器自動化工具模擬使用者操作（登入、填表、擷取資料）
- 了解 Docker 的基本概念，並學會安裝與啟動容器
- 認識 NemoClaw（NVIDIA 對 OpenClaw 的延伸產品），並完成基礎環境配置

核心概念說明

1. 為什麼需要 Web 自動化？

許多企業的重要資料仍然存放在各種網路平台上，例如：

- 政府開放資料平台
- 電商平台上的競品價格
- 產業研究報告網站
- 公司內部的 ERP / CRM 系統

這些資料通常**無法透過 API 直接取得**，只能透過「模擬人類操作瀏覽器」的方式來擷取。

---

2. Web 爬蟲 vs. Web 自動化

這兩個概念常被混用，但技術上有明確的差異：

	Web 爬蟲（Scraping）	**Web 自動化（Automation）**
核心目的	大量擷取網頁上的文字資料	模擬使用者在瀏覽器的操作行為
主要技術	requests + BeautifulSoup	Playwright、Selenium
能做什麼	讀取靜態網頁內容	點擊按鈕、填表、登入、處理動態頁面
適用情境	新聞抓取、靜態資料庫	需要登入的系統、動態載入的內容
限制	無法處理 JavaScript 動態內容	速度較慢，資源消耗較多

---

3. 瀏覽器自動化技術：Playwright

Playwright 是由 Microsoft 開發的現代瀏覽器自動化框架，能夠控制 Chrome、Firefox、Safari 等瀏覽器，執行各種使用者操作。

Playwright 能做到的事：

操作	**說明**

操作	說明
開啟網頁	導航至任意 URL
點擊元素	點擊按鈕、連結、核取方塊
填寫表單	輸入文字、選擇下拉選單
擷取資料	取得網頁上的文字、圖片、連結
截圖	對目前頁面截圖
等待元素	等到特定內容出現才繼續
處理彈出視窗	確認對話方塊、切換分頁
下載檔案	觸發下載並儲存到本機

認證流程自動化（重要概念）：

很多平台需要先登入才能存取資料。自動化登入的基本流程：

```
1. 開啟登入頁面
2. 找到「帳號」輸入框，填入帳號
3. 找到「密碼」輸入框，填入密碼
4. 點擊「登入」按鈕
5. 等待頁面跳轉完成
6. 確認登入成功（尋找登入後才有的元素）
7. 繼續執行後續任務
```

> **資安提醒：** 帳號密碼務必從環境變數讀取，絕對不能寫死在程式碼中，更不能上傳到 GitHub。

---

4. 網路反爬蟲機制與道德規範

技術面的挑戰：

許多網站會設置防護機制來阻擋自動化程式：

防護機制	說明
CAPTCHA	要求驗證使用者是人類
速率限制	太頻繁的請求會被封鎖
User-Agent 檢查	識別並封鎖非瀏覽器的請求
IP 封鎖	短時間大量請求會被封鎖 IP

道德與法律規範：

- 使用前請閱讀網站的**服務條款（Terms of Service）**，確認是否允許自動化存取
- 避免對目標網站造成不必要的伺服器負載（請求間隔應適當）
- 不應用於爬取受版權保護的內容或個人隱私資料

- 若網站提供 API，應優先使用 API 而非爬蟲

---

5. Docker 基礎概念

Docker 是一種「容器化」技術，可以把程式及其所有相依套件**打包在一個獨立、可攜帶的容器**中，確保在任何電腦上都能以完全相同的方式執行。

一個簡單的比喻：

想像 Docker 容器就像是一個**預先配置好環境的虛擬電腦：**

- 所有需要的軟體都已預先安裝
- 不會受到你電腦現有環境的干擾
- 在任何機器上執行結果都一樣

Docker 的核心概念：

概念	說明	生活比喻
Image（映像檔）	包含程式與環境的模板	食譜
Container（容器）	根據 Image 運行的實際程序	依照食譜做出來的料理
Registry	存放 Image 的倉庫	食譜書
Volume	容器與主機間的資料共享	共用材料箱

常用 Docker 指令（示意）：

```
# 下載 Image
docker pull [image名稱]

# 啟動容器
docker run [選項] [image名稱]

# 查看執行中的容器
docker ps

# 停止容器
docker stop [容器ID]
```

---

6. NemoClaw 簡介

NemoClaw 是 NVIDIA 基於 OpenClaw 框架延伸開發的 AI Agent 平台，針對 **GPU 加速運算**做了深度優化，特別適合：

- 需要大量本地 AI 推理的場景
- 處理敏感資料（無需將資料送至外部 API）
- 高效能的文件理解與多模態任務

NemoClaw 的主要優勢（相較於純 OpenClaw）：

特點	說明
本地 LLM 運行	不依賴外部 API，資料不離開本機
NVIDIA GPU 加速	大幅提升推理速度
企業級安全	適合處理機密或個資資料
多模態支援	可處理文字、圖片、PDF 等多種格式

NemoClaw 的安裝方式（透過 Docker）：

NemoClaw 提供官方 Docker 映像檔，透過 Docker 可以快速完成安裝，不需要複雜的環境配置，這也是本堂課選擇先介紹 Docker 的原因。

實作步驟（逐步教學）

實作一：安裝 Playwright 並執行第一個自動化腳本

步驟 1：安裝 Playwright

```
安裝 playwright 套件:
    pip install playwright

下載瀏覽器驅動程式:
    playwright install chromium
```

步驟 2：向 GitHub Copilot 描述你的第一個自動化需求

```
請幫我用 Python 和 Playwright 寫一個程式：
1. 開啟 Chrome 瀏覽器（有視覺畫面，不要用無頭模式）
2. 前往 https://www.google.com
3. 在搜尋框輸入「AI Agent 教學」
4. 點擊搜尋按鈕
5. 等待搜尋結果載入後，擷取前5個搜尋結果的「標題」和「網址」
6. 把結果列印在終端機上
```

步驟 3：觀察瀏覽器的操作過程

執行程式後，你會看到一個 Chrome 瀏覽器視窗自動開啟並完成搜尋，理解整個操作流程。

實作二：模擬登入某平台並擷取資料

> 本實作以一個**測試用的範例網站**進行演示（不針對任何特定商業平台）。

步驟 1：向 Copilot 描述需求

```
請幫我用 Playwright 寫一個自動登入程式：
1. 從環境變數讀取帳號（USERNAME）和密碼（PASSWORD）
2. 開啟目標網站的登入頁面
3. 找到帳號和密碼輸入框，填入認證資訊
4. 點擊登入按鈕
5. 等待跳轉，確認登入成功（檢查頁面標題或特定元素是否存在）
6. 導航到資料頁面，擷取表格資料
7. 把資料存成 CSV 檔案
請加入適當的等待機制和錯誤處理
```

步驟 2：理解「等待機制」的重要性

網頁通常需要一些時間才能載入完成，自動化程式必須「等待」元素出現才能操作：

等待類型	說明	適用場景
固定等待	強制等待固定秒數	簡單但效率低，不建議
等待元素出現	等到特定元素出現才繼續	最常用、最可靠
等待網路靜止	等到網頁不再有網路請求	初始頁面載入

實作三：安裝 Docker 並部署 NemoClaw

步驟 1：安裝 Docker Desktop

1. 前往 https://www.docker.com/products/docker-desktop/ 下載 Docker Desktop
2. 完成安裝後重新開機
3. 開啟 Docker Desktop，確認右下角顯示「Docker is running」

步驟 2：確認 Docker 正常運作

開啟終端機，輸入：

```
docker --version
docker run hello-world
```

若看到歡迎訊息，表示 Docker 安裝成功。

步驟 3：依照 NemoClaw 官方文件拉取映像檔

```
# 下載 NemoClaw 映像檔（依官方文件的實際指令執行）
docker pull nvcr.io/nvidia/nemoclaw:latest

# 啟動 NemoClaw 容器
docker run -d \
  --name nemoclaw \
```

```
  -p 8080:8080 \
  nvcr.io/nvidia/nemoclaw:latest
```

步驟 4：連接 NemoClaw 介面並設定工作流

1. 開啟瀏覽器，前往 `http://localhost:8080`
2. 完成初始化設定
3. 建立第一個 NemoClaw 工作流，連接剛才的 Web 自動化腳本作為工具節點

程式碼範例（虛擬碼）

Playwright 基礎自動化

```
載入 playwright 套件

# 啟動瀏覽器
啟動 playwright:
    瀏覽器 = 開啟Chromium(有視覺畫面=真)
    頁面 = 開新分頁()

    # 前往目標網址
    頁面.前往("https://目標網站.com")

    # 等待特定元素出現後操作
    頁面.等待元素("#帳號輸入框")
    頁面.填入("#帳號輸入框", 帳號)
    頁面.填入("#密碼輸入框", 密碼)
    頁面.點擊("#登入按鈕")

    # 等待登入完成
    頁面.等待URL改變()

    # 擷取資料
    資料表格 = 頁面.取得所有元素("table tr")
    對 每一列 在 資料表格 中:
        擷取每個欄位的文字
        加入資料清單

    # 儲存結果
    將資料清單寫入 CSV 檔案

    # 關閉瀏覽器
    瀏覽器.關閉()
```

含動態表單處理的進階腳本

```
函式 填寫搜尋條件(頁面, 搜尋參數):
    # 選擇下拉選單
```

```
    頁面.選擇("#地區選單", 搜尋參數.地區)

    # 填入日期範圍
    頁面.填入("#開始日期", 搜尋參數.開始日期)
    頁面.填入("#結束日期", 搜尋參數.結束日期)

    # 點擊搜尋
    頁面.點擊("#搜尋按鈕")

    # 等待結果載入（等待載入動畫消失）
    頁面.等待元素(".載入動畫", 狀態=消失)
    頁面.等待元素(".搜尋結果", 狀態=出現)


函式 擷取分頁資料(頁面):
    所有資料 = []

    # 處理多頁資料
    重複執行:
        當前頁資料 = 擷取表格資料(頁面)
        所有資料.加入(當前頁資料)

        下一頁按鈕 = 頁面.尋找元素(".下一頁")
        如果 下一頁按鈕 不存在 或 下一頁按鈕 被禁用:
            跳出迴圈
        下一頁按鈕.點擊()
        頁面.等待載入完成()

    回傳 所有資料
```

練習題 / 作業

課堂練習

練習 1：流程設計

設計一個「自動收集競品資訊」的 Web 自動化 Agent，描述以下內容：

- 目標平台是什麼類型的網站（電商、新聞、政府資料等）？
- 需要哪些操作步驟（不用寫程式碼，用文字描述）？
- 擷取到的資料要如何儲存和使用？

練習 2：識別等待時機

針對以下操作，判斷應該等待什麼條件才能繼續下一步：

操作	應等待的條件
點擊「搜尋」按鈕後	
點擊「下一頁」按鈕後	

操作	應等待的條件
點擊「下載」按鈕後	
填入搜尋關鍵字後	

課後作業

1. **Playwright 實作**：使用 Playwright 自動化擷取一個公開資料網站（例如政府開放資料平台）的特定資料，並儲存為 CSV 或 Excel 格式。
2. **Docker 熟悉**：在自己的電腦上成功安裝 Docker Desktop，並執行官方的 `hello-world` 容器，截圖留存。

延伸閱讀

Web 自動化工具：

- Playwright 官方文件 — https://playwright.dev/python/
- Playwright 選擇器指南 — 如何正確定位網頁元素

Docker 學習資源：

- Docker 官方入門教學 — https://docs.docker.com/get-started/
- 《Docker 深入淺出》— 友善的 Docker 入門書籍

NVIDIA NemoClaw：

- NVIDIA NIM 官方文件 — https://docs.nvidia.com/nim/（NemoClaw 的相關技術基礎）

法律與道德：

- 《網路爬蟲的法律灰色地帶》— 建議使用前進行法律諮詢
- Robots.txt 規範說明 — 了解 robots.txt 對自動化爬蟲的指引